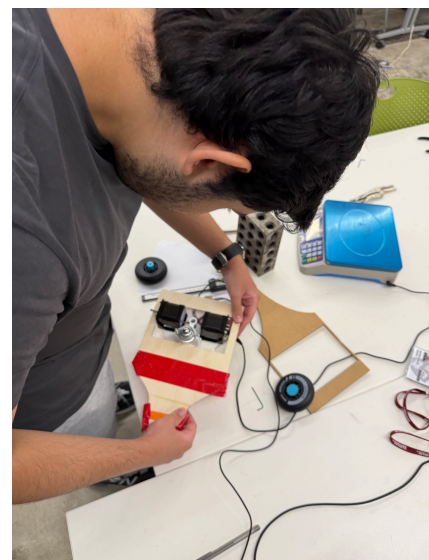
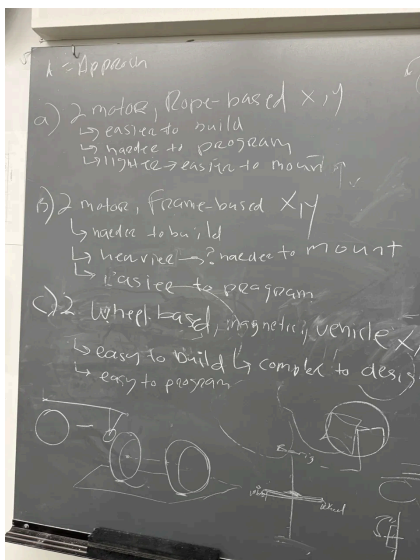


Explore Approaches for Machine Building; Prototype a 2-wheel magnetic vehicle X/Y; Pin Configuration of Existing Plotter Device

Exploration



Resources

- [PS70 Week 10](#) | Failed V-Plotter for PS70 '26? —
- [Youtube](#) | The Engineer who invented the Mars Rover Suspension...in his garage
- [Photos](#) | First Lab Session

Team

Arden, Gurmani, Matthew, Mert, Omer

Background

Our team needs to build something that uses two stepper motors to move along two dimensions for our machine-building project. We can choose the end effector of our choice (what is output from the X–Y movement) — for example, a plotter. After the assignment, our team was interested in building a vertical plotter.

Summary

We met to generate approaches for building a vertical plotter. Analyzing last semester's failed attempt (davidettel.com/PS70/week10.html) to build a vertical plotter for a glass surface with a marker as an end effector showed issues arising from (a) their mechanism — 4 stepper motors drawing a string — and (b) their end effector not having enough Z-axis force or rigidity to sustain the marker. The result was that they couldn't draw a circle.

We evaluated 3 approaches for a V-plotter on a chalkboard, with a magnet-based mount of varying types:

Approach	Pros	Cons
A) 2 motor, rope-based X/Y	• Easier to build; lighter → easier to mount	• Harder to program
B) 2 motor, frame-based X/Y	• Easier to program	• Harder to build; heavier → (likely) harder to mount
✓ C) 2-wheel magnetic vehicle X/Y	• Easy to build; easy to program	• Complex to design

We decided to attempt a **2-wheel magnetic vehicle X/Y** vertical plotter after testing the different devices (frame-based was ~3 kg and a single-magnet test showed promise for mounting to chalk; wheel-based was 1.6 kg and showed promise with a magnet holding test; vs. a string approach). We also decided we would 3D-print a chalk holder with a spring inside to apply constant pressure, and potentially position the chalk holder at an angle to reduce drag.

We found an existing 2x step-motor and wheel + ball-bearing chassis and protoboard.

Pin Direction of Existing Plotter Device

We also wrote some basic [code](#) to constantly accelerate both wheels.

Pins	Left Wheel	Right Wheel
------	------------	-------------

Step	D4	D9
Direction	D5	D7

Reflection

- Matt: The worst case scenario if we can't figure out the board mount, we can just have this be side sidewalk chalk plotter, then focus in on interesting input mechanisms (like a 2-rotary-encoder controller that uses ESPnow to communicate with the plotter like a remote control, or eventually gesture?)

Next Steps

- Create a magnet rig
 - Determine where magnet(s) are positioned to support the necessary force to hold the plotter upright, on wheels to drive coordinates
- Design software
 - Prove we can move in X–Y without falling
 - Determine if friction mitigates starting or stopping
 - Do we need constant micro-movement, or force counter-moving to prevent slip? Rotational issues? Do we need to alter the magnetic field somehow while moving?
- Design enclosure
 - Someone mentioned a car looking enclosure?
- Enhance input method (time based)
 - Level 1 | We'll start with a simple vector file for drawing coordinates so we have something that works
 - Level ? | 2× rotary encoders (acceleration vs. rotation, etch-a-sketch-style?)
 - Level ? | Gesture?
 - Surface based?
 - Air based?
 - Level ? | Voice?

Appendix

Code

```
#include <AccelStepper.h>

const int stepPin1 = D4; // LEFT WHEEL
const int dirPin1 = D5;
```

```
const int stepPin2 = D9;
const int dirPin2 = D7;

const int NUM_STEPPERS = 2;

AccelStepper steppers[NUM_STEPPERS] = {
    AccelStepper(1, stepPin1, dirPin1),
    AccelStepper(1, stepPin2, dirPin2)
};

void setup()
{
}

int spinDirection = 1;

void loop()
{
    for (int currentStepper = 0; currentStepper < NUM_STEPPERS - 1;
currentStepper++) {
        spinDirection = spinDirection * -1;
        int maxSpeed = 4000 * spinDirection;
        int speed = 4000 * spinDirection;
        AccelStepper &stepper = steppers[currentStepper];
        stepper.setMaxSpeed(maxSpeed);
        stepper.setSpeed(speed);
        stepper.runSpeed();
    }
}
```